

SPEKIFIKASI API DIGIMENU

1. Pendahuluan

Dokumen ini menjelaskan endpoint yang tersedia pada sistem pemesanan restoran berbasis Laravel 11. Sistem digunakan secara *onsite* (pelanggan memesan via menu digital, kasir mengelola meja, admin mengelola data master).

2. Standar Teknis

- **Base URL:** http://127.0.0.1:8000
- **Format Data:** application/json (untuk endpoint proses), HTML (untuk halaman view)
- **Autentikasi:** Session-based (Laravel auth middleware) untuk area Admin
- **Framework:** Laravel 11

3. Kode Status Respons

- 200 OK – Permintaan berhasil
- 302 Redirect – Redirect setelah proses (login, store, update, delete)
- 401/403 – Tidak memiliki akses (belum login / bukan admin)
- 404 Not Found – Data tidak ditemukan
- 422 – Validasi gagal
- 500 – Kesalahan server

4. Ringkasan Endpoint

A. Halaman Pelanggan (Public)

Method	Endpoint	Controller/Closure	Deskripsi
GET	/	Closure	Halaman utama, menampilkan daftar meja & status (tables)
GET	/menu	Closure	Menampilkan daftar produk (products) & nomor order otomatis (orderNumber, format #ORD-0001)
GET	/payment	Closure	Halaman pembayaran
POST	/process-checkout	OrderController@store	Memproses checkout/pesanan baru
GET	/order/{id}/receipt	Closure	Menampilkan struk digital berdasarkan id order

B. Area Kasir

Method	Endpoint	Controller	Deskripsi
GET	/kasir/meja	CashierController@index	Menampilkan daftar meja untuk

				kasir
POST	/kasir/meja/toggle/{id}	CashierController@toggleStatus	Mengubah status meja (kosong/terisi)	

C. Autentikasi Admin

Method	Endpoint	Controller	Name Route	Deskripsi
GET	/admin/login	AuthController@showLoginForm	admin.login / login	Form login admin
POST	/admin/login	AuthController@login	admin.login.submit	Proses login
POST	/admin/logout	AuthController@logout	admin.logout	Logout admin

D. Area Admin (Protected – middleware auth)

Method	Endpoint	Controller	Name Route	Deskripsi
GET	/admin/dashboard	AdminDashboardController@index	admin.dashboard	Dashboard admin
GET	/admin/menu	AdminMenuController@index	admin.menu	Daftar menu/produk
POST	/admin/menu/store	AdminMenuController@store	admin.menu.store	Tambah produk baru
PUT	/admin/menu/update/{id}	AdminMenuController@update	admin.menu.update	Update produk
DELETE	/admin/menu/delete/{id}	AdminMenuController@destroy	admin.menu.destroy	Hapus produk
GET	/admin/table	AdminTableController@index	admin.table	Daftar meja
POST	/admin/table/store	AdminTableController@store	admin.table.store	Tambah meja baru
PUT	/admin/table/update/{id}	AdminTableController@update	admin.table.update	Update data meja
DELETE	/admin/table/delete/{id}	AdminTableController@destroy	admin.table.destroy	Hapus meja
GET	/admin/export-excel	ReportController@exportExcel	admin.export	Export laporan ke Excel

5. Detail Endpoint

5.1 GET /

Deskripsi: Mengambil seluruh data meja (Table::all()), digunakan untuk menampilkan status meja (merah = terisi, hijau = kosong) di halaman utama.

Response (view: welcome) — asumsi struktur, lihat catatan pada 5.9:

```
{
  "tables": [
    { "id": 1, "name": "A1", "status": "kosong" },
    { "id": 2, "name": "A2", "status": "terisi" }
  ]
}
```

5.2 GET /menu

Deskripsi: Menampilkan semua produk dan menghitung nomor order berikutnya berdasarkan ID order terakhir.

Response (view: menu):

```
{
  "products": [
    {
      "id": 1,
      "name": "Nasi Goreng",
      "price": 25000.00,
      "category": "makanan",
      "image_url": "products/nasi-goreng.jpg",
      "is_spicy": true,
      "is_gluten_free": false,
      "created_at": "2026-06-01T08:00:00.000000Z",
      "updated_at": "2026-06-01T08:00:00.000000Z"
    }
  ],
  "orderNumber": "#ORD-0005"
}
```

5.3 POST /process-checkout

Endpoint: OrderController@store

Deskripsi: Menyimpan pesanan baru beserta item-itemnya. Satu order memiliki banyak order_items (relasi one-to-many), dan setiap order_item terhubung ke satu product.

Request Body:

```
{
  "table_id": 1,
  "guest_count": 2,
  "items": [
    { "product_id": 1, "quantity": 2, "price": 25000 },
    { "product_id": 3, "quantity": 1, "price": 12000 }
  ]
}
```

Validasi:

| Field | Aturan |

|---|---|

| table_id | nullable, harus ada di tabel tables (exists:tables,id) |

| guest_count | required, integer, min:1 |

| items | required, array, min:1 |

| items.*.product_id | required, harus ada di tabel products (exists:products,id) |

| items.*.quantity | required, integer, min:1 |

| items.*.price | required, integer, min:0 |

Proses di server:

1. total_amount dihitung otomatis dari penjumlahan quantity * price seluruh item (tidak diterima dari client untuk menghindari manipulasi harga).
2. Simpan record baru ke tabel orders dengan status = 'pending'.
3. Simpan setiap item ke tabel order_items dengan order_id yang baru dibuat.

Response Success (201 Created):

```
{
  "status": "success",
  "order_id": 5,
}
```

```
"total_amount": 62000,  
"redirect": "/order/5/receipt"  
}
```

Response Error – Validasi Gagal (422 Unprocessable Entity):

```
{  
  "status": "error",  
  "message": "Validasi gagal",  
  "errors": {  
    "items": ["Item pesanan tidak boleh kosong."],  
    "items.0.quantity": ["Jumlah item minimal 1."]   
  }  
}
```

Response Error – Server (500):

```
{  
  "status": "error",  
  "message": "Terjadi kesalahan saat menyimpan pesanan."  
}
```

5.4 GET /order/{id}/receipt

Deskripsi: Menampilkan struk digital berdasarkan ID pesanan, termasuk relasi `order_items` dan masing-masing product.

Response (view: receipt):

```
{  
  "order": {  
    "id": 5,  
    "table_id": 1,  
    "guest_count": 2,  
    "total_amount": 62000.00,  
    "status": "pending",  
    "created_at": "2026-06-10T10:00:00.000000Z",  
    "order_items": [  
      {  
        "id": 11,  
        "order_id": 5,  

```

```
{
  "product_id": 1,
  "quantity": 2,
  "price": 25000,
  "product": {
    "id": 1,
    "name": "Nasi Goreng",
    "category": "makanan"
  }
},
{
  "id": 12,
  "order_id": 5,
  "product_id": 3,
  "quantity": 1,
  "price": 12000,
  "product": {
    "id": 3,
    "name": "Es Teh Manis",
    "category": "minuman"
  }
}
],
},
"orderNumber": "#ORD-0005"
}
```

Response Error – Order Tidak Ditemukan (404):

```
{
  "status": "error",
  "message": "Order tidak ditemukan."
}
```

Catatan: Saat ini route menggunakan `Order::findOrFail($id)` tanpa `eager-load` relasi (`with('orderItems.product')`). Disarankan menambahkan `eager-load` agar struk dapat menampilkan detail item dan nama produk tanpa `N+1` query.

5.5 GET /kasir/meja

Endpoint: `CashierController@index`

Deskripsi: Menampilkan daftar meja beserta statusnya untuk dikelola kasir.

5.6 POST /kasir/meja/toggle/{id}

Endpoint: CashierController@toggleStatus

Deskripsi: Mengubah status meja (toggle antara kosong dan terisi) berdasarkan id meja.

Response:

```
{
  "status": "success",
  "table_id": 1,
  "new_status": "terisi"
}
```

5.7 Autentikasi Admin

POST /admin/login

Request Body:

```
{
  "email": "admin@resto.com",
  "password": "password123"
}
```

Response Success: Redirect ke /admin/dashboard

Response Gagal: Redirect kembali ke /admin/login dengan pesan error validasi.

POST /admin/logout

Deskripsi: Menghapus session login admin, redirect ke /admin/login.

5.8 Manajemen Menu (Admin)

POST /admin/menu/store

Request Body (multipart/form-data):

```
{
  "name": "Es Teh Manis",
  "price": 5000.00,
  "category": "minuman",
  "image_url": "(file upload)",
  "is_spicy": false,
  "is_gluten_free": true
}
```

Validasi:

| Field | Aturan |

|---|---|

| name | required, string, max:255 |

| price | required, numeric, min:0 |

| category | nullable, string, max:100 |

| image_url | nullable, image (jpg, jpeg, png), max:2048KB |

| is_spicy | boolean |

| is_gluten_free | boolean |

Response Error – Validasi Gagal (422):

```
{
  "status": "error",
  "errors": {
    "name": ["Nama produk wajib diisi."],
    "price": ["Harga harus berupa angka."]
  }
}
```

PUT /admin/menu/update/{id}

Request Body: sama seperti store, field yang ingin diubah saja (gunakan `_method=PUT` jika dikirim via form HTML).

Response Error – Data Tidak Ditemukan (404):

```
{
  "status": "error",
  "message": "Produk tidak ditemukan."
}
```

DELETE /admin/menu/delete/{id}

Deskripsi: Menghapus produk berdasarkan id.

Response Success:

```
{
  "status": "success",
  "message": "Produk berhasil dihapus."
}
```

5.9 Manajemen Meja (Admin)

Catatan: Migration untuk tabel `tables` belum dilampirkan. Field di bawah ini adalah **asumsi** berdasarkan penggunaan kolom `status` pada `CashierController` dan `Order::table_id` (foreign key ke `tables.id`). Mohon sesuaikan dengan migration asli.

POST /admin/table/store

Request Body (asumsi):

```
{
  "name": "A5",
  "status": "kosong"
}
```

Validasi (asumsi):

| Field | Aturan |

|---|---|

| name | required, string, max:50, unique:tables,name |

| status | required, in:kosong,terisi |

PUT /admin/table/update/{id}

Request Body: sama seperti store.

DELETE /admin/table/delete/{id}

Deskripsi: Menghapus data meja berdasarkan id. Karena orders.table_id menggunakan onDelete('cascade'), menghapus meja akan **ikut menghapus seluruh order yang terkait dengan meja tersebut**. Pertimbangkan untuk mengganti relasi menjadi onDelete('set null') jika data riwayat order ingin tetap tersimpan meski meja dihapus.

Response Error – Data Tidak Ditemukan (404):

```
{
  "status": "error",
  "message": "Meja tidak ditemukan."
}
```

5.10 GET /admin/export-excel

Endpoint: ReportController@exportExcel

Deskripsi: Mengunduh laporan transaksi dalam format .xlsx.

Response: File Excel (Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet)

6. Catatan

- Endpoint pada bagian C dan D bersifat *web route* (mengembalikan view/redirect), bukan murni RESTful JSON API. Jika ingin dijadikan API murni untuk aplikasi mobile/eksternal, disarankan membuat grup route baru di bawah prefix /api menggunakan routes/api.php dengan response JsonResponse dan autentikasi token (Sanctum).
- Struktur field pada contoh JSON (nama_produk, harga, dll.) perlu disesuaikan dengan nama kolom asli pada migration/model masing-masing (Product, Table, Order).